System Programming Project 3

담당 교수 : 이영민

이름 : 강유영

학번 : 20231508

1. 개발 목표

본 프로젝트의 목적은 여러 클라이언트가 동시에 주식 서버에 접속하여 주식 정보를 조회하고 매매할 수 있는 서버 프로그램을 구현하는 것이다. 서버는 기본적인 CRUD가 아닌 동시성 처리, I/O 멀티플렉싱 또는 멀티스레딩 기반의 구조를 통해 효율적으로 여러 요청을 처리할 수 있어야 한다. 또한 종료 시 주식 정보가 파일에 저장되어야 하며, 성능 비교를 통해 두 방식의 차이점과 장단점을 분석하는 것이 목적이다.

2. 개발 범위 및 내용

A. 개발 범위

1. Task 1: Event-driven Approach

select()를 활용한 이벤트 기반 서버를 구현하였다. 단일 프로세스 내에서 select를 통해 다수의 클라이언트 요청을 I/O Multiplexing 방식으로 처리하였다.

2. Task 2: Thread-based Approach

pthread를 이용한 스레드 기반 서버를 구현하였다. 메인 스레드는 연결 요청을 큐에 추가하고, 고정된 개수의 워커 스레드가 이를 처리하였다.

3. Task 3: Performance Evaluation

두 방식에 대해 클라이언트 수 및 요청 유형에 따라 성능 차이를 측정하고 분석하였다. client가 show만 할 경우, buy와 sell만 할 경우, 셋 다 할 경우로 워크로드를 나누었으며 각 경우에 client 수가 4부터 8, 12, 16, 20 이렇게 5가지 경우로 나누어서 성능 차이를 비교하였다.

B. 개발 내용

- **Task1 (Event-driven Approach with select())**

✓ Multi-client 요청에 따른 I/O Multiplexing 설명

select()를 사용하여 listen socket과 연결된 모든 클라이언트 fd를 동시에 감시하며, I/O 이벤트가 발생한 클라이언트만 처리하는 방식이다. 각 클라이언트의 fd는 pool 구조체에 등록되며, main 루프에서 select()를 호출하여 읽기 가능한 fd 집합을 확인한다. 해당 fd에 데이터가 도착하면 해당 클라이언트로부터 요청을 읽고 처리한 뒤 응답을 보낸다. 이를 통해 단일 스레드로도 여러 클라이언트의 요청을 병렬적으로 처리할 수 있다.

✓ epoll과의 차이점 서술

select는 모든 fd 집합을 순회해야 하며, 감시할 수 있는 최대 fd 수(FD_SETSIZE)에 제한이 있다. 또한 모든 호출마다 전체 fd 집합을 다시 전달해야 하므로 오버헤드가 크다. 반면 epoll은 커널 수준에서 이벤트를 비동기적으로 감지하여 필요한 fd만 알려주기 때문에 성능이 우수하고, 수천 개 이상의

fd를 효율적으로 감시할 수 있다. 따라서 select는 소규모 연결에서는 간단하지만, 대규모 연결 처리에는 epoll이 더 적합하다.

- **Task2 (Thread-based Approach with pthread)**

  ✓ Master Thread의 Connection 관리

Main 함수에서 Master Thread는 클라이언트의 연결 요청을 처리하는 역할을 수행한다. listenfd를 통해 수신 대기 상태를 유지하다가, Accept 함수를 통해 클라이언트가 접속하면 해당 클라이언트의 연결 소켓 디스크립터(connfd)를 내부에 정의된 연결 큐(ConnQueue)에 enqueue 한다. 연결 수락과 enqueue 작업은 순차적으로 이루어지며, 이 작업이 완료되면 worker thread가 이를 처리할 수 있도록 대기 상태를 깨운다. Master Thread는 실제 요청 처리를 담당하지 않고, 오직 연결 수락과 큐 등록을 수행하여 역할을 분리함으로써 처리 부하를 줄이고 스레드 간 병렬성을 높인다.

  ✓ Worker Thread Pool 관리하는 부분에 대해 서술

프로그램 시작 시 THREAD_COUNT 수만큼 pthread_create를 통해 worker thread를 생성한다.각 thread는 프로그램 종료 시까지 무한 루프를 돌며 연결 큐에서 하나의 connfd를 dequeue 한 뒤, serve() 함수를 통해 해당 클라이언트의 요청을 처리한다. 각 worker는 pthread_detach를 호출하여 자원 회수를 자동으로 수행하도록 설정된다. serve() 내부에서는 클라이언트로부터 명령어(show, buy, sell, exit)를 수신하고, 적절한 함수 호출과 응답 처리를 진행한다. 모든 요청 처리가 끝나면 해당 클라이언트의 연결을 종료하고, stock.txt를 갱신한다. 이러한 방식은 고정된 수의 스레드를 유지 하면서도 많은 수의 요청을 병렬로 처리할 수 있도록 해준다.

- **Task3 (Performance Evaluation)**

  ✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

본 실험에서 측정하고자 하는 주요 성능 지표는 서버 처리 소요 시간이다. 이는 multiclient 프로그램에서 모든 클라이언트 프로세스를 fork하기 직전 시점을 기준으로, 모든 자식 프로세스가 종료되어 부모 프로세스에서 waitpid로 회수된 직후까지의 시간 차이를 측정한 것이다. 측정은 gettimeofday() 함수를 이용하여 초 단위가 아닌 마이크로초(us) 단위의 정밀한 성능 비교가 가능하도록 하였다.

이 metric을 선택한 이유는, 서버의 전체 처리 시간이 실제 서비스 환경에서의 응답 대기 시간과 처리량에 직접적인 영향을 주기 때문이다. 또한 해당 프로젝트에서는 단일 서버 프로세스에 다수의 클라이언트 요청이 동시 도달하는 상황을 시뮬레이션하고 있기 때문에, 전체 소요 시간을 통해 각 서버 구조의 확장성과 효율성을 직관적으로 비교할 수 있다.

  ✓ Configuration 변화에 따른 예상 결과 서술

성능 비교 실험에서는 다음의 두 가지 구성을 바탕으로 서버 구조(Task 1: select, Task 2: thread)의 차이를 평가한다.

클라이언트 수 변화 : 클라이언트 수를 4, 8, 12, 16, 20으로 증가시켜가며 서버의 확장성을 분석한다. 예상되는 결과는 다음과 같다.

> Task 1 (select 기반) : 클라이언트 수 증가에 따른 처리 시간 변화가 작을 것으로 예상된다. select는 단일 쓰레드 기반이며, 클라이언트 소켓을 multiplexing 하므로 context switching이나 락 충돌이 발생하지 않기 때문이다.

Task 2 (pthread 기반) : 클라이언트 수가 증가함에 따라 처리 시간이 선형 또는 그 이상으로 증가할 것으로 예상된다. 다수의 스레드가 공유 자원에 접근하면서 세마포어에 의한 동기화 충돌이 발생하고, context switching 오버헤드가 누적되기 때문이다.

워크로드 유형 변화:

A. 모든 클라이언트가 show만 요청 : 동기화가 거의 필요하지 않아 두 방식 간 차이는 작을 것으로 예상된다. 하지만 thread 방식은 쓰레드 수 증가에 따른 관리 오버헤드로 인해 select 방식보다 더 많은 시간을 소요할 가능성이 있다.

B. 모든 클라이언트가 buy 또는 sell 만 요청 : select 방식은 직렬 처리로 인해 처리 시간이 안정적으로 유지되지만, thread 방식은 락 경합(lock contention)으로 인해 성능이 크게 저하될 것으로 예상된다.

C. 모든 클라이언트가 show, buy, sell 섞어서 요청 : 실제 서비스와 유사한 workload로, 위 두 방식의 특성이 모두 반영된다. 이 경우에도 select 방식이 더 일정한 처리 성능을 보일 것으로 예상된다.

C. 개발 방법

Task1 구현을 위해 stockserver.c에 select 기반의 I/O 멀티플렉싱 구조를 도입하였다. 클라이언트 fd들을 추적하기 위해 pool_t 구조체를 정의하였고, read_set과 ready_set을 사용하여 클라이언트들의 이벤트를 감지하였다. main 루프에서는 Select()를 호출하여 이벤트 발생 fd를 확인하고, handle_client() 함수를 통해 해당 클라이언트 요청을 처리하였다. handle_client()는 show, buy, sell, exit 명령어를 파싱하여 각각의 기능을 수행하며, 처리 후 stock.txt에 변경사항을 반영한다. 주식 정보는 BST로 관리하며, 각 노드에는 readcnt, mutex, rw_mutex를 포함하여 readers-writers locking 기법을 적용하였다.

Task2 구현을 위해 stockserver.c에 연결 큐(ConnQueue) 구조체를 정의하고 생산자-소비자 모델로 구현하였다. 메인 스레드는 Accept()로 클라이언트 연결을 수락하고 해당 connfd를 큐에 enqueue하며, 워커 스레드는 큐에서 fd를 꺼내 요청을 처리한다. pthread를 이용해 고정 개수의 스레드를 생성하고, 각 스레드는 serve() 함수를 통해 명령어를 처리하고 stock.txt를 갱신한다. BST 구조는 Task1과 동일하며, fine-grained locking을 통해 각 노드 단위로 동기화하였다. SIGINT 발생 시 모든 변경사항을 파일에 저장하고 안전하게 종료된다.

Task3 구현을 위해 multiclient.c 파일을 수정하여 성능 평가가 가능하도록 측정 기능을 추가하였다. gettimeofday() 함수를 통해 성능 측정을 위한 시작 시점과 종료 시점을 기록하며, fork 호출 직전과 모든 자식 프로세스의 종료 직후에 시간을 측정하였다. 워크로드 유형에 따라 multiclient에서 클라이언트가 show 명령만 수행하도록 하거나 buy/sell 명령만 수행하도록 조건문을 구성하고, 다양한 클라이언트 수(예: 4, 8, 12, 16, 20)로 실험을 반복할 수 있도록 유연한 구조를 구성하였다. show 명령어의 경우 서버로부터 여러 줄의 응답이 오므로 readline을 통해 응답을 줄 단위로 수신하고 출력하며, buy/sell 명령은 동기화가 필요한 연산이므로 성능에 영향을 줄 수 있는 요소로 실험 대상이 된다. 각 실험 결과는 Elapsed time: ... us 형식으로 출력되며, 이를 통해 Task1과 Task2의 처리 시간 및 확장성 차이를 정량적으로 비교할 수 있다.

3. 구현 결과

Task1과 Task2 모두 정상적으로 구현되어 4명 이하 클라이언트 환경에서 정상적으로 동작하였다. 명령어(show, buy, sell, exit)에 따라 서버가 적절한 응답을 반환하고, 종료 후 stock.txt에 내용이 갱신되는 것도 확인하였다.

Task3에서는 multiclient에 성능 측정 기능을 추가하고 다양한 클라이언트 수와 워크로드 유형에 따라 실험을 수행하였다. 측정 결과는 예상대로 Task1은 일정한 처리 시간을, Task2는 클라이언트 수에 따라 선형 증가하는 경향을 보였다. 구현은 정상 작동하였으며, 예외 상황 처리 및 리소스 정리에 대한 보완은 향후 개선 계획으로 고려 중이다.

미처 구현하지 못한 부분은 클라이언트 수가 매우 클 경우의 graceful shutdown 및 메모리 해제 루틴이며, 이는 추가 구현 계획으로 고려 중이다.

4. 성능 평가 결과 (Task 3)

4-1. 기본 설정 (전역 변수)

```
#define MAX_CLIENT        100
#define ORDER_PER_CLIENT  10
#define STOCK_NUM         10
#define BUY_SELL_MAX      10
```

4-2. 기본 설정 (측정 시작, 종료 시점)

측정 시작 시점: multiclient에서 모든 클라이언트 프로세스를 fork하기 직전

측정 종료 시점: multiclient에서 모든 클라이언트 프로세스가 종료한 직후

4-3. 성능 평가

A. 모든 클라이언트가 show만 요청

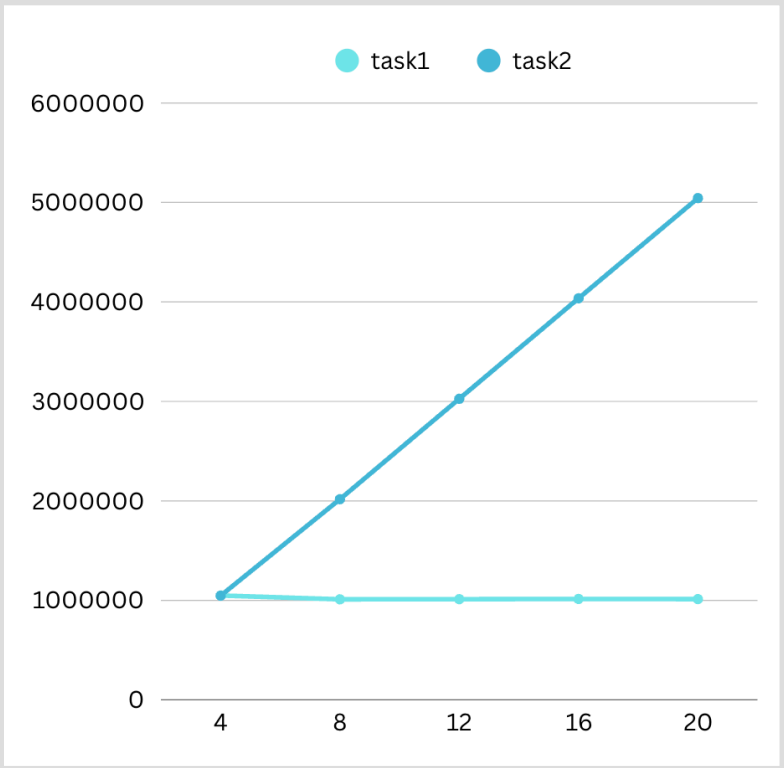
client 수	task1 측정 시간 (us)	task2 측정 시간 (us)
4	1048310	1046199
8	1009795	2016383
12	1011280	3026249
16	1013204	4036910
20	1011957	5044091

```
cse20231508@cspro8:~/20231508/task1$ ./multiclient 172.30.10.8 60207 4
Elapsed time: 1048310 us
cse20231508@cspro8:~/20231508/task1$ ./multiclient 172.30.10.8 60207 8
Elapsed time: 1009795 us
cse20231508@cspro8:~/20231508/task1$ ./multiclient 172.30.10.8 60207 12
Elapsed time: 1011280 us
cse20231508@cspro8:~/20231508/task1$ ./multiclient 172.30.10.8 60207 16
Elapsed time: 1013204 us
cse20231508@cspro8:~/20231508/task1$ ./multiclient 172.30.10.8 60207 20
Elapsed time: 1011957 us
```

(A-task 1 성능 측정 터미널)

```
cse20231508@cspro8:~/20231508/task2$ ./multiclient 172.30.10.8 60207 4
Elapsed time: 1046199 us
cse20231508@cspro8:~/20231508/task2$ ./multiclient 172.30.10.8 60207 8
Elapsed time: 2016383 us
cse20231508@cspro8:~/20231508/task2$ ./multiclient 172.30.10.8 60207 12
Elapsed time: 3026249 us
cse20231508@cspro8:~/20231508/task2$ ./multiclient 172.30.10.8 60207 16
Elapsed time: 4036910 us
cse20231508@cspro8:~/20231508/task2$ ./multiclient 172.30.10.8 60207 20
Elapsed time: 5044091 us
```

(A-task 2 성능 측정 터미널)



모든 클라이언트가 show 명령만 요청한 경우, 클라이언트 수에 따라 Task1과 Task2의 처리 시간을 측정한 결과를 비교한 그래프를 분석해보면 다음과 같다. Task1(select 기반)은 클라이언트 수가 증가해도 처리 시간이 약 1,000,000us 내외로 안정적으로 유지되며, 클라이언트 수의 증가가 서버 처

리 시간에 거의 영향을 주지 않았다. 반면, Task2(pthread 기반)는 클라이언트 수가 늘어날수록 처리 시간이 선형적으로 증가하여 4명일 때는 1,046,199us이었지만, 20명에서는 5,044,091us로 약 5배 이상 증가하였다. 이 차이는 두 구조의 병렬 처리 방식의 차이에서 비롯되며, select 기반은 단일 프로세스가 I/O 멀티플렉싱을 통해 효율적으로 다수의 클라이언트를 처리하는 반면, thread 기반은 각 요청을 별도의 워커 스레드가 처리하는 구조로 인해 스레드 간 컨텍스트 스위칭 및 관리 오버헤드가 누적되기 때문이다. 특히 show 명령은 읽기 작업으로 동기화가 필요하지 않기 때문에 select 방식의 구조가 더 높은 효율성을 보인다. 따라서 읽기 중심의 워크로드 환경에서는 select 기반 구조가 더 나은 확장성과 처리 성능을 제공할 수 있음을 실험을 통해 확인할 수 있다.

B. 모든 클라이언트가 buy 또는 sell만 요청

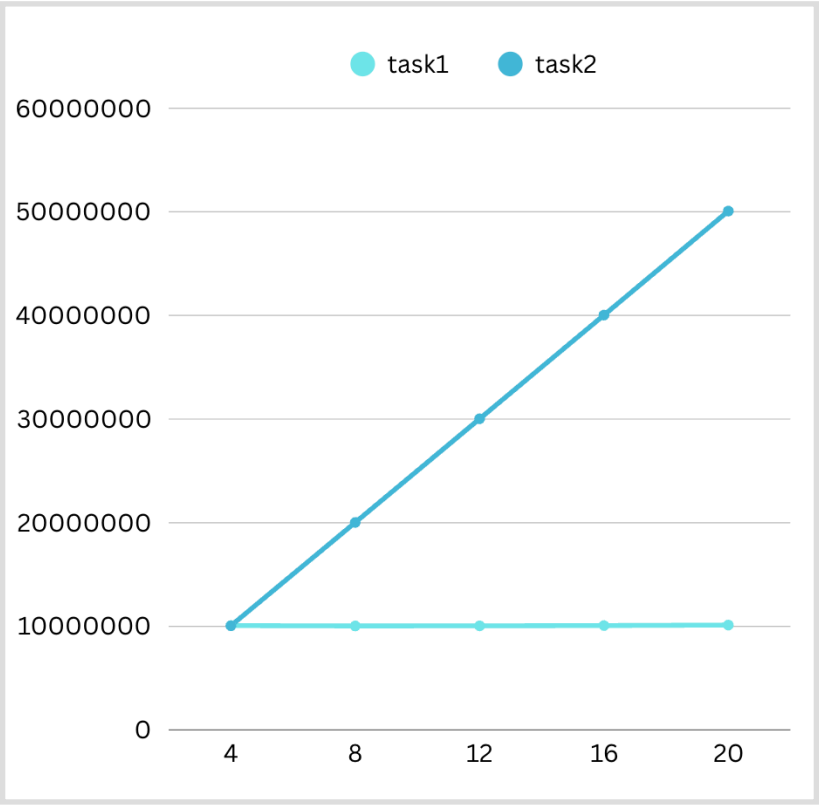
client 수	task1 측정 시간 (us)	task2 측정 시간 (us)
4	10066782	10049234
8	10031741	20018127
12	10044011	30023848
16	10068205	40037002
20	10118435	50080584

```
[buy] success
[buy] success
Elapsed time: 10049234 us
cse20231508@cspro8:~/20231508/task
[buy] success
[sell] success
Elapsed time: 20018127 us
cse20231508@cspro8:~/20231508/task
[buy] success
Elapsed time: 30023848 us
cse20231508@cspro8:~/20231508/task
child 1869884
[buy] success
[buy] success
Elapsed time: 40037002 us
cse20231508@cspro8:~/20231508/task
child 1869934
[buy] success
[buy] success
Elapsed time: 50080584 us
cse20231508@cspro8:~/20231508/task
```

(B-task 1 성능 측정 터미널)

```
[buy] success
[sell] success
Elapsed time: 10066782 us
cse20231508@cspro8:~/20231508/tas
[buy] success
[buy] success
Elapsed time: 10031741 us
cse20231508@cspro8:~/20231508/tas
child 1869631
[sell] success
[sell] success
Elapsed time: 10044011 us
cse20231508@cspro8:~/20231508/tas
[sell] success
Elapsed time: 10068205 us
cse20231508@cspro8:~/20231508/tas
child 1869690
[sell] success
[buy] success
Elapsed time: 10118435 us
cse20231508@cspro8:~/20231508/tas
```

(B-task 2 성능 측정 터미널)



모든 클라이언트가 buy 또는 sell 명령만 요청하는 경우의 실험 결과를 보면, Task1(select 기반)과 Task2(pthread 기반)의 성능 차이가 더욱 극명하게 드러난다. Task1은 클라이언트 수가 증가함에도 불구하고 전체 처리 시간이 약 10,000,000us 수준에서 일정하게 유지되는 반면, Task2는 클라이언트 수가 증가할수록 처리 시간이 선형적으로 증가하여 4명일 때는 약 10,049,234us였으나 20명에

서는 무려 50,080,584us로 5배 이상 늘어났다. 이 결과는 쓰기 연산이 포함된 buy/sell 명령이 주식 정보를 수정하며, 이에 따라 서버 내부의 동기화가 필수적으로 발생하기 때문이다. Task1에서는 요청이 직렬로 처리되어 동기화 충돌이 없고, 처리 속도는 느리지만 예측 가능하게 유지된다. 반면, Task2는 병렬 처리를 시도하지만, 쓰레드 간 공유 자원 접근으로 인한 세마포어 충돌과 컨텍스트 스위칭 오버헤드가 누적되며 성능 저하가 발생한다. 따라서 쓰기 중심의 워크로드에서는 단순 병렬화가 반드시 성능 향상을 보장하지 않으며, 오히려 안정적인 직렬 처리 구조가 더 나은 효율을 제공할 수 있음을 이 실험 결과는 보여준다.

C. 모든 클라이언트가 show, buy, sell 섞어서 요청

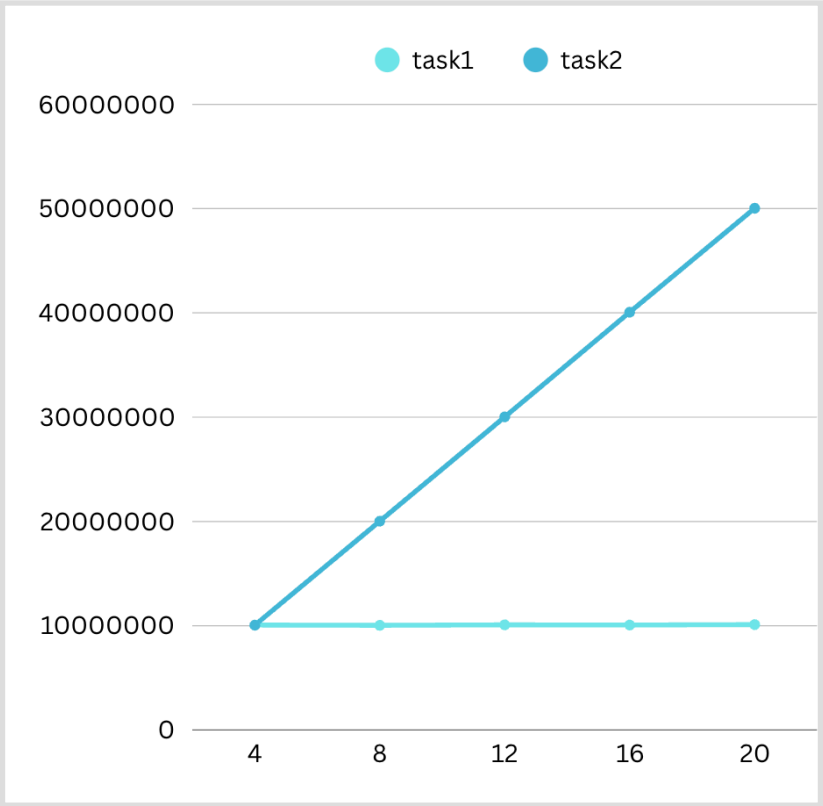
client 수	task1 측정 시간 (us)	task2 측정 시간 (us)
4	10060352	10046692
8	10033125	20018179
12	10078025	30025779
16	10061682	40071346
20	10100970	50043266

```
9 920 5000
10 993 5500
Elapsed time: 10060352 us
cse20231508@cspro8:~/20231508/tas
9 925 5000
10 995 5500
Elapsed time: 10033125 us
cse20231508@cspro8:~/20231508/tas
10 1001 5500
[sell] success
Elapsed time: 10078025 us
cse20231508@cspro8:~/20231508/tas
[buy] success
[buy] success
Elapsed time: 10061682 us
cse20231508@cspro8:~/20231508/tas
[buy] success
[buy] success
Elapsed time: 10100970 us
cse20231508@cspro8:~/20231508/tas
```

(C-task 1 성능 측정 터미널)

```
[buy] success
[buy] success
Elapsed time: 10046692 us
cse20231508@cspro8:~/20231508/tasl
9 893 5000
10 1019 5500
Elapsed time: 20018179 us
cse20231508@cspro8:~/20231508/tasl
[sell] success
[sell] success
Elapsed time: 30025779 us
cse20231508@cspro8:~/20231508/tas
[sell] success
[sell] success
Elapsed time: 40071346 us
cse20231508@cspro8:~/20231508/tas
10 1024 5500
[sell] success
Elapsed time: 50043266 us
cse20231508@cspro8:~/20231508/task
```

(C-task 2 성능 측정 터미널)



클라이언트가 show, buy, sell 명령을 섞어서 요청한 경우의 성능 측정 결과를 분석해보면, Task1(select 기반)은 클라이언트 수가 증가해도 처리 시간이 약 10,000,000us 수준으로 일정하게 유지되며 안정적인 성능을 보인다. 반면, Task2(pthread 기반)는 클라이언트 수가 늘어날수록 처리

시간이 선형적으로 증가하여 4명에서는 10,046,692us였던 것이 20명에서는 50,043,266us까지 증가하였다. 이는 혼합 워크로드에서 읽기 요청(show)은 별도의 동기화가 필요 없지만, 쓰기 요청(buy/sell)은 동기화가 필요하기 때문에 Task2에서는 여러 워커 스레드가 동시에 동일 자원(BST 주식 노드)에 접근하면서 세마포어 충돌이 발생하고 그에 따른 병목 현상이 발생했기 때문이다. 반면, Task1은 하나의 프로세스가 순차적으로 모든 요청을 처리하기 때문에 처리 속도는 느릴 수 있으나 안정적이고 예측 가능한 성능을 유지한다. 결과적으로 show, buy, sell이 혼합된 실제 서비스에 가까운 환경에서도 select 기반 구조(Task1)가 더 높은 확장성과 안정적인 처리 성능을 제공하는 것을 확인할 수 있다.

4-4. 최종 평가

Task3에서는 event-driven 방식(Task1)과 thread-based 방식(Task2)의 서버 구조에 대해, 클라이언트 수와 워크로드 유형에 따라 처리 성능을 비교 평가하였다. 성능 측정은 multiclient 프로그램에 gettimeofday()를 활용하여 fork 직전과 자식 프로세스 종료 직후를 기준으로 측정된 elapsed time을 사용하였다. 클라이언트 수는 4, 8, 12, 16, 20으로 점차 증가시키며 실험을 진행하였고, 워크로드 유형은 show-only, buy/sell-only, 혼합 요청(show + buy + sell)으로 구분하여 각각 테스트하였다. show-only 워크로드에서는 Task1과 Task2 모두 비교적 빠른 응답을 보였으나, Task2는 클라이언트 수가 증가할수록 처리 시간이 선형적으로 증가하였고, Task1은 안정적인 처리 시간을 유지하였다. buy/sell-only 워크로드에서는 쓰기 연산에 따른 세마포어 충돌로 인해 Task2는 병목이 심화되어 클라이언트 수가 증가할수록 성능 저하가 급격하게 나타났으며, 반면 Task1은 직렬 처리임에도 처리 시간이 일정하게 유지되었다. 혼합 워크로드에서는 두 구조의 특성이 동시에 반영되며 Task2는 동기화 비용에 의한 성능 저하가 그대로 유지되었고, Task1은 일관된 처리 성능을 보였다. 전체 실험 결과는 수업 시간에 배운 I/O 멀티플렉싱 방식(select)의 확장성과 안정성, 그리고 thread 기반 구조의 병렬성 장점과 동시에 발생 가능한 동기화 충돌과 context switching 오버헤드의 한계를 그대로 반영하였다. 결론적으로 Task1은 안정적인 처리 성능을 보이며 확장성이 뛰어나고, Task2는 클라이언트 수가 적을 때는 병렬 처리의 이점을 보이지만, 클라이언트 수 증가 시 병목이 발생하여 오히려 비효율적일 수 있음을 확인할 수 있었다. 이러한 분석은 단순히 성능 수치를 비교하는 것을 넘어, 실질적인 서버 설계 시 선택할 구조의 기준을 성능, 확장성, 동기화 비용 관점에서 판단할 수 있게 해준다.